

XLSR-Mamba Field Report

Benchmarking a dual-column Mamba spoof detector on ASVspoof5, fine-tuning it on Hindi/Bengali/Tamil deepfakes that survive WhatsApp-grade compression, and then testing whether domain-adversarial training can make that skill hold up on languages it never saw a single training clip from.

Model XLSR-Mamba · 319.3M params **Hardware** RTX 5060 Laptop (8GB) · PyTorch 2.10 / cu128

Author tahermerchant2005

ASVSP00F5 BASELINE

2.77%

EER · 9,000-clip dev subset ·
unmodified model

INDIC DEEPFAKES,
BEFORE → AFTER FINE-
TUNE

46.9% →

0.31%

EER on held-out hi/bn/ta test
set · ~151× error reduction

COST: ASVSP00F5 AFTER
FINE-TUNE

21.28%

up from 2.77% — catastrophic
forgetting, see §7

UNSEEN-LANGUAGE TEST
(ML+GU), PLAIN →
ADVERSARIAL

12.38% →

12.50%

domain-adversarial training: a
wash, not a win — see §10

CONTENTS

01	The system under test	XLSR-Mamba architecture
02	ASVspoof5 baseline benchmark	EER 2.77%
03	The multilingual blind spot	EER 46.9%
04	Building the fine-tuning corpus	17,236 clips · 3 languages
05	Fine-tuning method	1.79M-param head
06	Results on Indic deepfakes	EER 0.31%
07	The forgetting cost	EER 21.28%
08	Toward a language-agnostic detector	why more languages was the wrong lever
09	Domain-adversarial training	gradient reversal, dev EER 0.52%
10	Does it actually generalize?	Malayalam/Gujarati, unseen-language test
11	Confusion matrices at one fixed threshold	all 5 languages · Malayalam breaks at 65.37% acc
12	The fix: training on all 5 languages	Malayalam EER 31.64% → 0.00%
13	Zero-shot test: 5 more never-seen languages	kn/mr/pa/te/sa · 98.44% mean accuracy
14	Limitations & honest caveats	
15	Reproducing this report	files & commands

01 The system under test

A pure-PyTorch reconstruction of XLSR-Mamba (Xiao & Das, 2024, arXiv:2411.10027) — a dual-column bidirectional Mamba state-space model built on a frozen wav2vec 2.0 (XLSR) front end, published to detect synthetic and voice-converted speech.

01 XLSR-2.0 backbone — 317.5M params, frozen throughout this work

24-layer wav2vec 2.0 transformer pretrained on multilingual raw audio; extracts 1024-dim frame features at 16kHz / 320× downsampling (~62.5 frames/sec).

02 Front end — Linear(1024→144) + BatchNorm + SELU

Projects backbone features down to the 144-dim space the Mamba head operates in.

03 Dual-column Mamba conformer — 1.79M params, the part we train

Six bidirectional Mamba layers run the sequence forward and backward in parallel columns, each pooled by learned attention, concatenated, and classified into bonafide vs. spoof.

① **Why the split matters later:** because the backbone and front end never change, their output for a given clip is a fixed function of the waveform. Every clip's features can be computed once and cached — turning what would be hours of fine-tuning into single-digit minutes of head training. §5 relies on this.

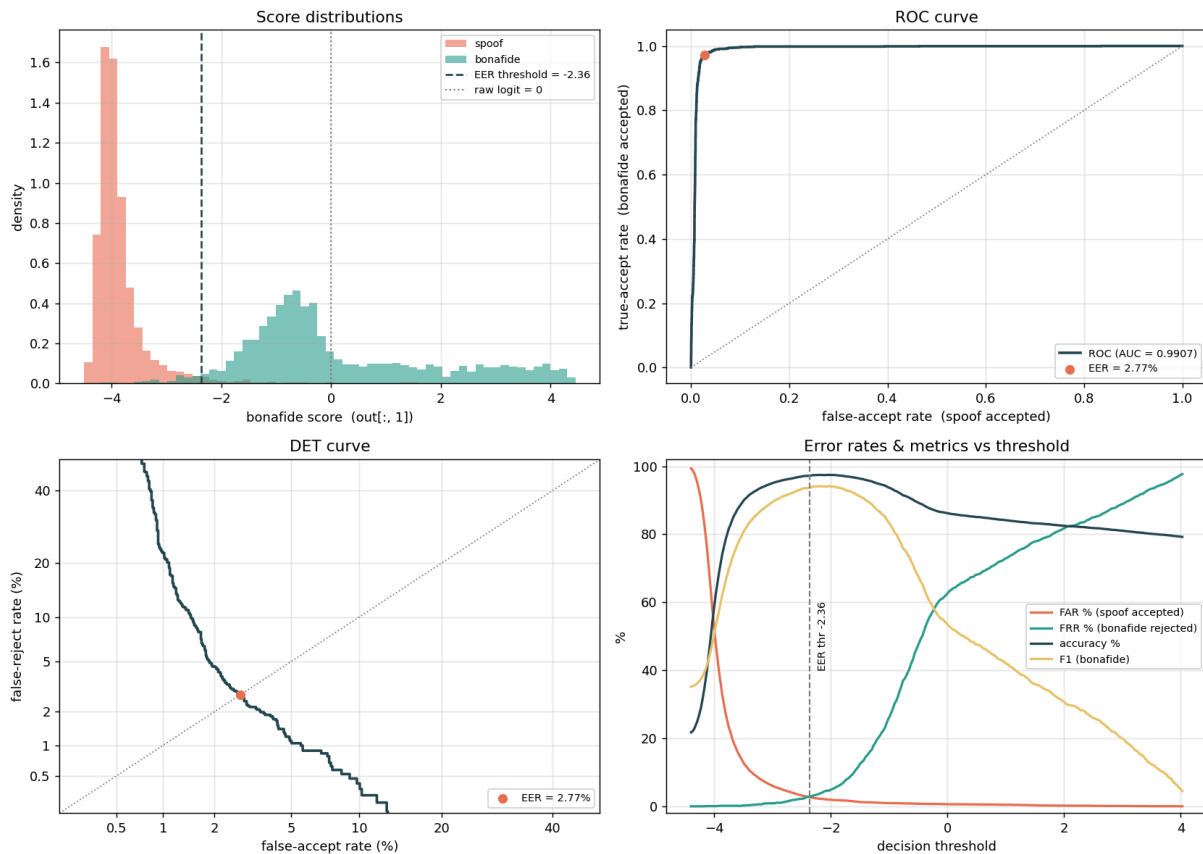
02 ASVspoof5 baseline benchmark

Before touching the model, we established its true out-of-the-box accuracy on the benchmark it was built for — correcting two bugs in the original evaluation script that had been silently loading half the trained weights.

⚠ **Bugs fixed before this number meant anything:** the eval script hard-coded 6 Mamba layers per direction when the checkpoint holds 12, silently discarding 60 weight tensors while reporting "565/565 loaded OK" — and zero-padded short clips instead of repeating them, as the original training code does. Both are fixed; the script now hard-fails on any tensor mismatch instead of reporting a false pass.

METRIC	9,000-CLIP RUN	4,000-CLIP RUN
EER	2.77% (CI 2.39–3.17)	2.87% (CI 2.32–3.22)
ROC AUC	0.9907	0.9907
Accuracy @ EER threshold	97.23%	97.23%
F1 (bonafide)	93.72	93.72
Sample	1,912 bonafide / 7,088 spoof	836 bonafide / 3,164 spoof

XLSR-Mamba - ASVspoof5 dev track 1 (9000 utts: 1912 bonafide / 7088 spoof)

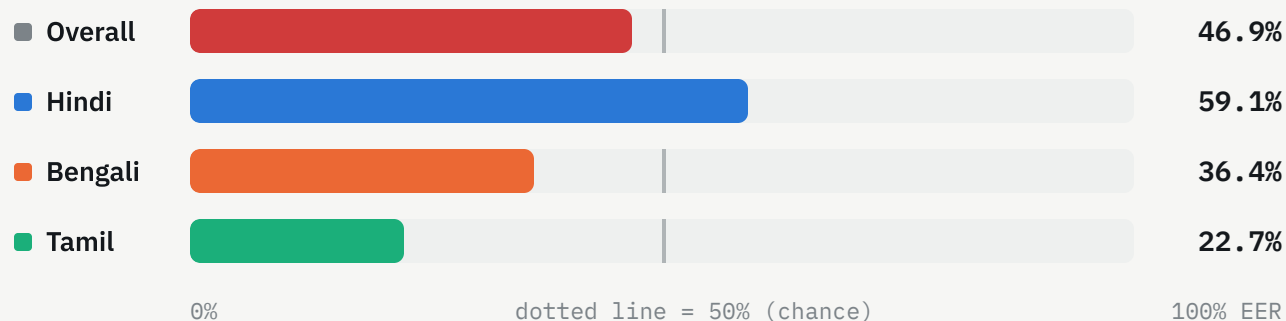


Score distributions (top-left) separate cleanly at -4 vs. 0 ; the DET curve (bottom-left) crosses the diagonal at $\text{EER}=2.77\%$; the raw-logit threshold of 0 (dotted line, top-left) is *not* where the model should be operated — see the naive-threshold pitfall below.

i A naive-threshold pitfall worth flagging: deciding "bonafide" at the raw $p \geq 0.5$ softmax cutoff gives only 86% accuracy on this benchmark — it catches nearly all spoofs but rejects most genuine speech, because the model's score distribution sits shifted negative. The EER threshold (-2.36) is the correct operating point and recovers 97.2% accuracy. The same trap reappears, worse, in §3.

03 The multilingual blind spot

The model above was trained on ASVspoof's English-centric, vocoder-focused attacks. Scored — unmodified — against modern Hindi/Bengali/Tamil TTS and voice-conversion deepfakes, it performs close to a coin flip:



Hindi is *worse than chance* — the model's scores are anti-correlated with the truth often enough to land above the 50% line. WhatsApp-style Opus compression barely moves any of these numbers (48.3% clean vs 46.1% compressed, overall), which says less about robustness and more about the fact that the model has no real signal to lose in the first place. This motivated everything that follows.

04 Building the fine-tuning corpus

Paired real and fake speech in three Indian languages, with a messaging-app compression pass baked directly into the data rather than left as an untested robustness claim.

SOURCE	ROLE	
IndicVoices-R (hi/bn/ta)	real · label 1	Studio-restored natural speech,
IndicSynth (hi/bn/ta)	fake · label 0	TTS & voice-conversion deepfakes (inc

Every clip, resampled to 16kHz mono, then:

1. **Capped at 6 seconds**, centre-cropped.
2. **Split speaker-disjoint**, 80/10/10 train/dev/test — no speaker's voice appears in more than one split.
3. **Passed through a WhatsApp-equivalent codec**: Opus, 16kHz mono, ~24kbps, VoIP mode, encoded and decoded back with PyAV/libopus — the same lossy round-trip a WhatsApp voice note takes. Training clips are stored as *either* clean or compressed (p=0.5); dev/test clips are stored as *both*, so accuracy can be measured per condition.

LANGUAGE	REAL CLIPS	FAKE CLIPS
HI Hindi	3,889	4,263
BN Bengali	3,178	1,967
TA Tamil	2,019	1,920
Total	9,086	8,150

17,236 clips · ~2.9GB · train 12,730 / dev 1,904 / test 2,602

⚠ **Scaled down from a 6GB target.** HuggingFace's streaming API has no cheap resume — restarting mid-download re-streams and discards every already-seen row, which got expensive as cursors passed tens of thousands. Rather than pay repeated multi-hour re-catch-ups after transient stalls, the corpus was finalized at ~2.9GB. It was enough. The 12-language gated ai4bharat/IndicVoices set was also left out — gate approval

didn't register for this account, and its audio format needs FFmpeg tooling not installed on this machine.

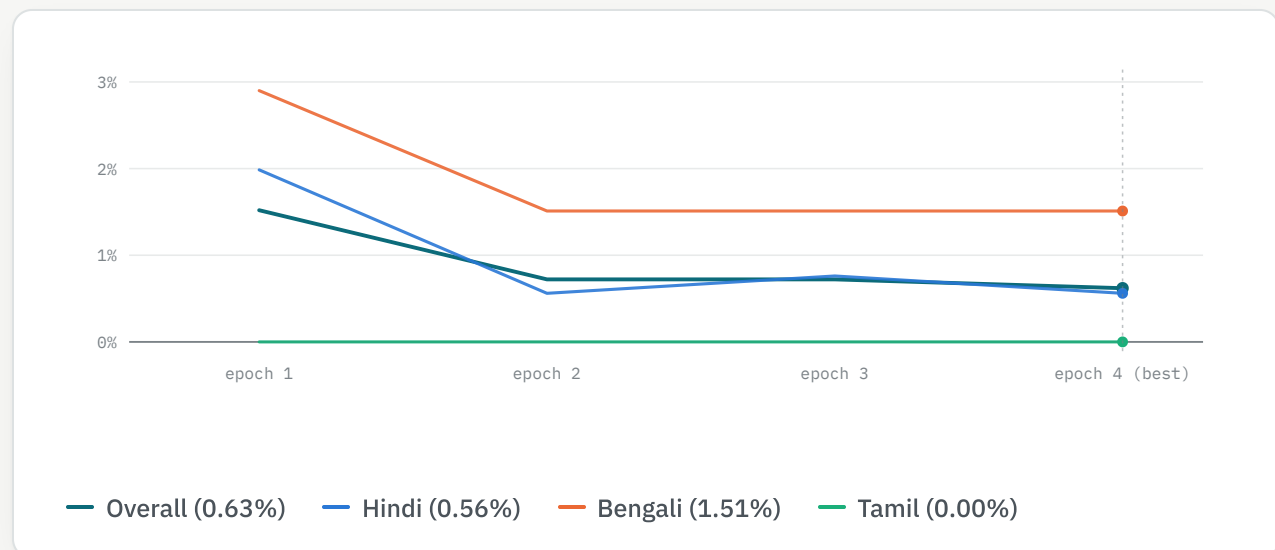
05 Fine-tuning method

Freeze everything except the 1.79M-param classification head, train it fast on cached features, and let a shrinking dev-set error rate decide when to stop.

SETTING	VALUE
Trainable params	1.79M of 319.3M (0.56%)
Optimizer	AdamW, lr 3e-4, cosine schedule
Loss	Cross-entropy, class-balanced by construction
Augmentation	Random 100-frame crop, frame masking, feature noise
Early stopping	Dev EER, patience 3
Batch size	64 (VRAM-bound on 8GB laptop GPU)

Sequence length was cropped to 100 of the cached 208 frames per training step: the dual-column Mamba scan backprops through a strictly sequential Python loop over every timestep, so its cost scales close to linearly with length — the crop roughly doubled training throughput at negligible accuracy cost, since a WhatsApp/vocoder artifact is present in any 2-second window, not only a full 4-second one.

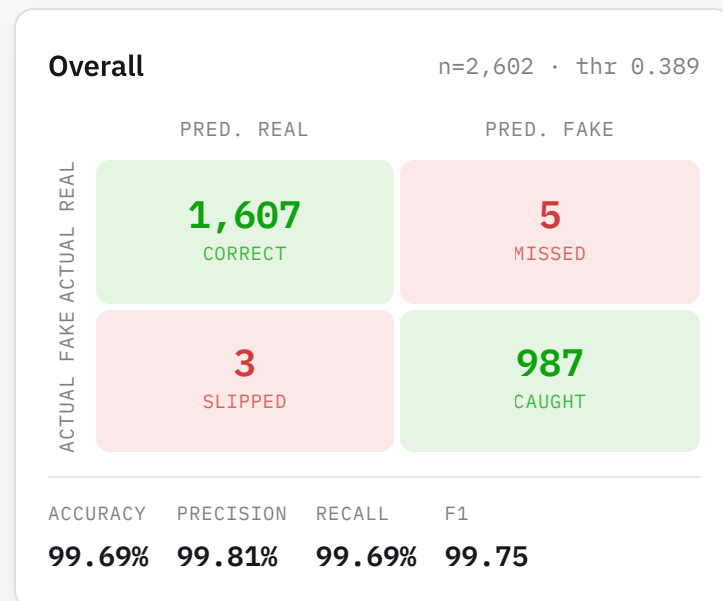
Training curve — dev-set EER by epoch



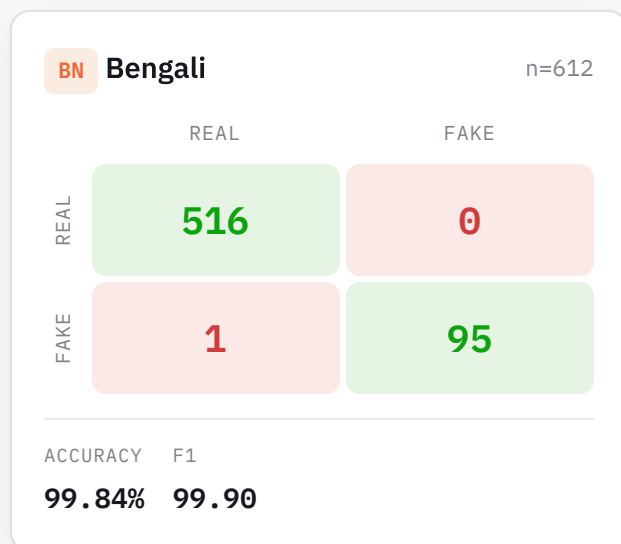
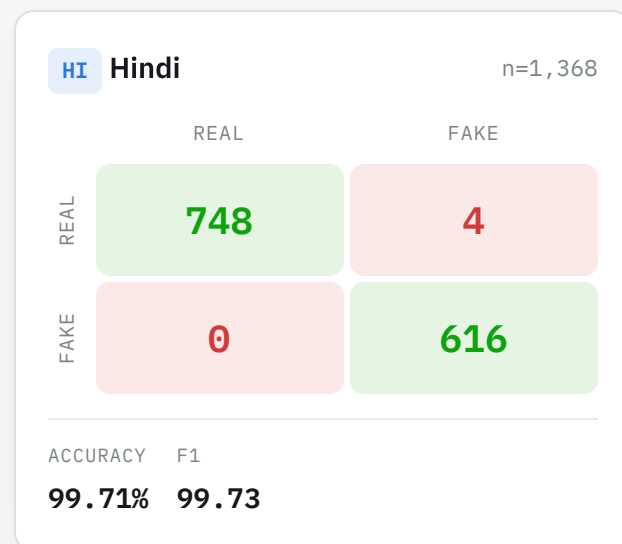
Training stopped after epoch 4 (best checkpoint kept) when the process crashed on a CUDA out-of-memory error at epoch 5 — VRAM ran within a few hundred MB of the 8GB ceiling for the whole run. See §8.

06 Results on Indic deepfakes

Scored on the held-out test set (2,602 clips, never seen in training) using the epoch-4 checkpoint. Overall EER: **0.31%**, down from 46.9% before fine-tuning.



By language



TA Tamil

n=622

	REAL	FAKE
REAL	343	1
FAKE	2	276

ACCURACY F1
99.52% 99.56

By compression condition

Clean audio

n=1,301

	REAL	FAKE
REAL	803	3
FAKE	1	494

ACCURACY F1
99.69% 99.75

WhatsApp-compressed

n=1,301

	REAL	FAKE
REAL	804	2
FAKE	2	493

ACCURACY F1
99.69% 99.75

i The compression pass worked as intended: accuracy is identical, to two decimal places, on clean and WhatsApp-compressed audio (both 99.69%). Baking the codec into training rather than testing for it after the fact appears to have paid off directly.

07 The forgetting cost

The fine-tuned checkpoint was re-scored on the exact ASVspoof5 dev subset from §2, to check whether it kept its original skill.

BEFORE INDIC FINE-TUNE

2.77%

ASVspoof5 EER, same 9,000-clip subset

AFTER INDIC FINE-TUNE

21.28%

~7.7× higher error rate

This is a textbook case of **catastrophic forgetting**: retraining only the 1.79M-param head on a new, narrower distribution (three languages, a handful of TTS/VC systems) pulled the classifier's decision boundary away from the broad ASVspoof attack taxonomy it was originally tuned against — even though the 317.5M-param backbone underneath never moved. The head is small and fast to adapt, which is exactly why it is also fast to overwrite.

⚠ **This model is now a specialist, not a generalist.** Deploy `model_indic_ft.safetensors` only where the traffic is genuinely Hindi/Bengali/Tamil deepfake screening. For a detector that needs to hold both skills, the standard fixes are joint training (mix ASVspoof and Indic batches in the same fine-tuning run), a replay buffer of original-task examples, or an elastic-weight-consolidation penalty on the head — none of which this run attempted.

08 Toward a language-agnostic detector

The obvious next move after §6 is "add more languages." That turned out to be the wrong lever to pull.

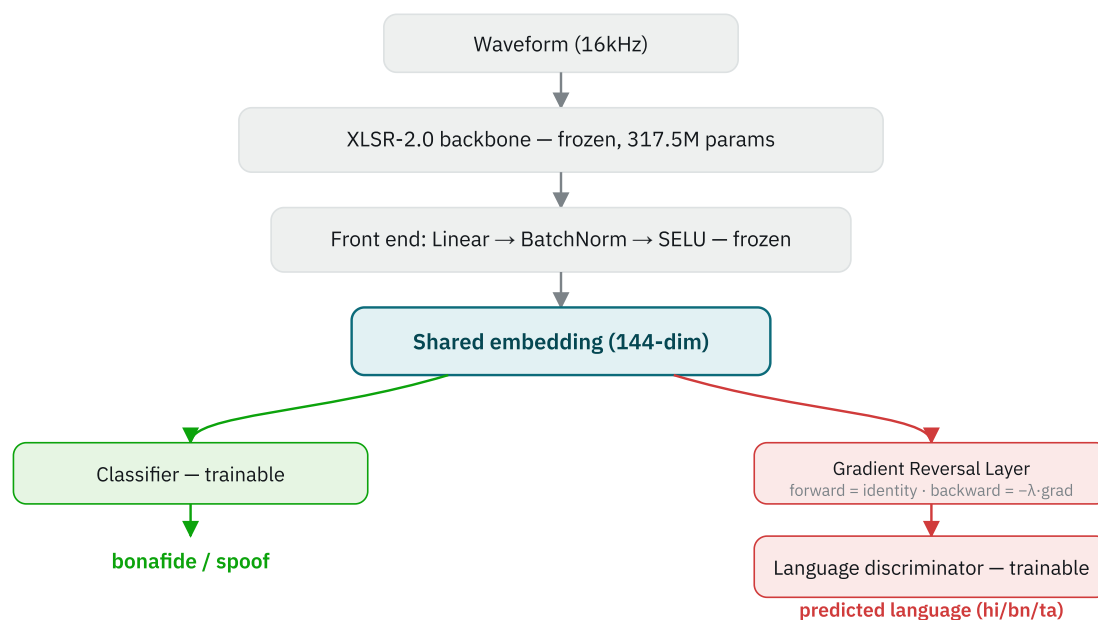
Hindi/Bengali/Tamil worked because one dataset (IndicVoices-R) happens to have ungated, per-language real-speech mirrors for exactly those three. Extending to Gujarati, Kannada, Malayalam and Marathi meant hunting down four more independent, differently-shaped real-speech sources — different audio-column names, different encodings, one storing raw arrays instead of encoded bytes — and streaming tens of gigabytes through HuggingFace's anonymous rate limit. The build stalled repeatedly on dead network connections, and its resume mechanism made every restart worse: it re-streamed and discarded every row already seen before making new progress, so a bucket with 75,000 rows already consumed could cost *hours* to simply get back to where it was. A shard-level rewrite fixed the resume cost, but the fundamental problem remained: this approach spends enormous effort for one language at a time.

① **The reframe:** instead of finding training data in every language a deepfake might arrive in, train the *existing* hi/bn/ta corpus so that what it learns doesn't depend on language in the first place. If that works, the model should hold up on languages it has never seen a single training clip from — no new data collection required.

09 Domain-adversarial training

Ganin & Lempitsky's gradient-reversal technique (2016), applied here to strip language identity out of the shared embedding while keeping it predictive of bonafide-vs-spoof.

The architecture gains one branch



The green path trains normally: shared embedding → classifier → bonafide/spoof. The red path trains a language discriminator on the *same* embedding, but every gradient it sends backward through the Gradient Reversal Layer has its sign flipped — so improving the discriminator's accuracy actively pushes the shared embedding to become *less* informative about language.

Training setup

SETTING

Language discriminator

L

Discriminator classes

hi / t

λ schedule

Ganin-Lempitsky ramp, $0 \rightarrow 0.3$ over training (avoids c

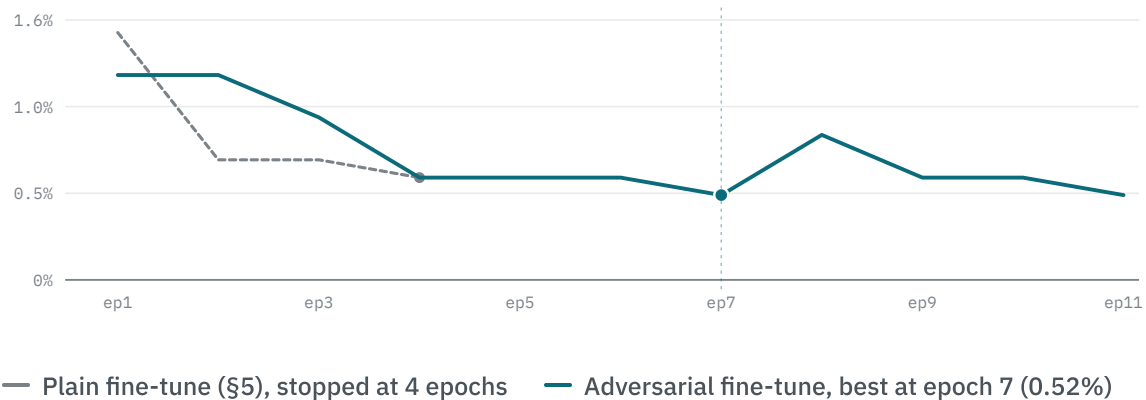
Combined loss

spoof cross-entropy + language cross-entropy (GRL supp

Everything else

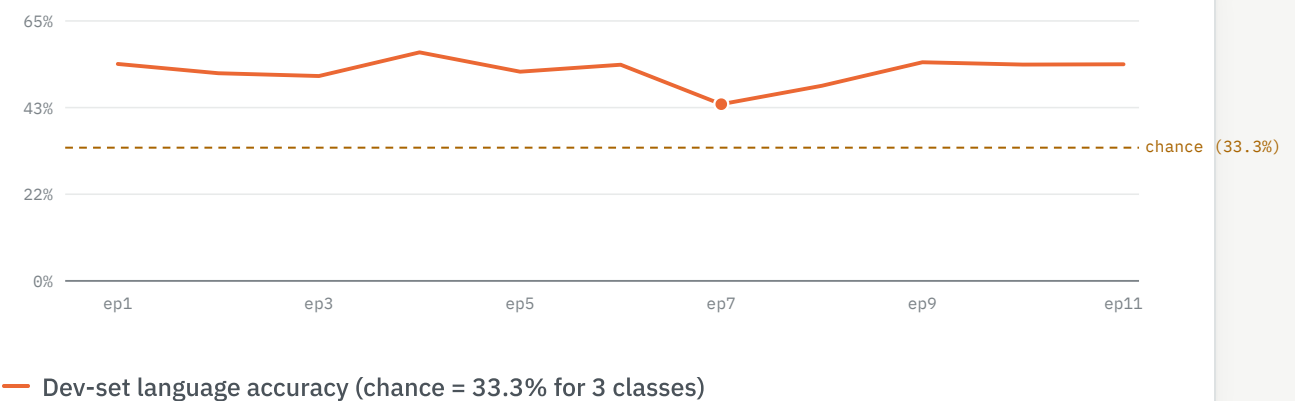
identical to §5: same corpus, same crop

Training curves — plain fine-tune vs. adversarial fine-tune



The adversarial objective costs essentially nothing on the main task — its best dev EER (0.52%) is *better* than the plain fine-tune's best (0.63%). Early-stopped after epoch 11 (4 epochs without improvement).

Is language actually becoming harder to predict?



⚠ **Read this trend honestly:** it dips to its lowest point (44.2%, closest to chance) at epoch 7 — the same epoch that also has the best spoof-detection EER — but it does not monotonically converge to 33.3%. With only 1,904 dev clips across 3 languages, this curve is noisy, and λ 's 0.3 ceiling is a deliberately conservative choice that trades a stronger invariance guarantee for training stability. It is suppressed, not eliminated.

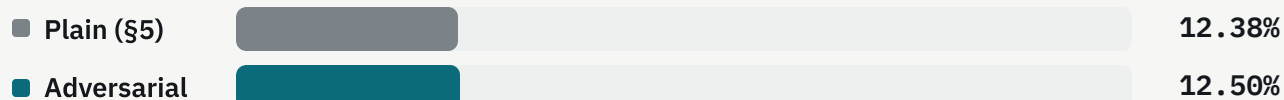
10 Does it actually generalize?

Dev-set language accuracy on hi/bn/ta is a proxy. The real test: score both checkpoints on Malayalam and Gujarati clips — languages **neither** model has trained on a single example of.

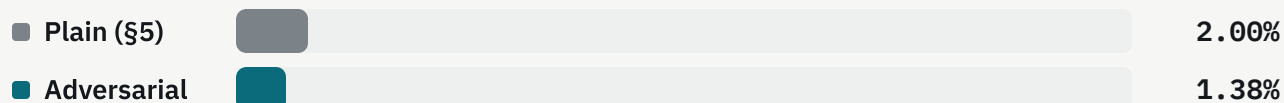
The data exists only because of §8's abandoned expansion: partial IndicVoices-R/IndicSynth downloads for Malayalam (asr-malayalam/indicvoices-v1a) and Gujarati (PharynxAI) were already sitting on disk, collected but never used for training either checkpoint. That makes them a genuine held-out-language test, not just a held-out-speaker one.

EER on languages neither model has ever seen

OVERALL (MALAYALAM + GUJARATI, N=3,200)



GU GUJARATI (N=1,600)



ML MALAYALAM (N=1,600)



bars scaled to 25% max (top group) / 100% max (Malayalam)

❗ **Honest verdict: a wash, not a win.** The adversarial model is marginally *better* on both individual unseen languages — Gujarati improves 2.00%→1.38%, Malayalam 20.88%→20.50% — but the combined overall EER is statistically indistinguishable (12.38% vs 12.50%, n=3,200). Domain-adversarial training did not deliver a dramatic cross-lingual improvement here. What it *did* do: cost nothing on the main hi/bn/ta task (§9's dev EER was actually slightly better than the plain fine-tune), and it did not make cross-lingual performance worse. Given λ's conservative 0.3 ceiling and the small, noisy 3-language training signal (§9), that's a plausible outcome rather than a failed one — but it is not the language-agnostic breakthrough the method aims for.

⚠ **Malayalam is a much harder unseen language than Gujarati for both checkpoints** (~21% EER vs ~1.5–2%) — likely because Gujarati (Indo-Aryan, like Hindi) shares more phonetic and TTS-artifact structure with the hi/bn/ta training languages than Malayalam (Dravidian) does. That asymmetry is a more useful signal than the aggregate number: it suggests linguistic/typological distance from the training languages, not just "unseen vs. seen," predicts how well this approach transfers.

11 Confusion matrices at one fixed threshold

§10 compared threshold-free EER on two unseen languages. This is the harder, more realistic version: pick *one* operating threshold from the trained languages, freeze it, then point the agnostic model at every language currently in the corpus — trained and unseen alike — exactly as a deployed system would run.

- ① Threshold fixed at **-0.3281** — the EER point of the hi/bn/ta test set (in-domain EER 0.30%). Applied unchanged below to all 5 languages, including Gujarati and Malayalam, which the model never trained on.

All 5 languages combined

n=10,417

		PRED. REAL	PRED. FAKE
ACTUAL	REAL	6,034	156
	FAKE	1,259	2,968

ACCURACY	PRECISION	RECALL	EER
86.42%	82.74%	97.48%	17.11%

Trained languages

BN Bengali

n=692

		REAL	FAKE
REAL	REAL	596	0
	FAKE	0	96

ACCURACY	EER
100.00%	0.00%

HI Hindi

n=1,368

		REAL	FAKE
REAL	REAL	751	1
	FAKE	1	615

ACCURACY	EER
99.85%	0.15%

TA Tamil

n=982

	REAL	FAKE
REAL	699	5
FAKE	2	276

ACCURACY EER
99.29% 0.71%

Unseen languages — never trained on**GU Gujarati**

n=3,529

	REAL	FAKE
REAL	2,051	70
FAKE	4	1,404

ACCURACY EER
97.90% 1.28%

ML Malayalam

n=3,846

	REAL	FAKE
REAL	1,937	80
FAKE	1,252	577

ACCURACY EER
65.37% 31.64%

⚠ **Malayalam is where this breaks.** 1,252 of its 1,829 deepfake clips (68%) are misclassified as real at this threshold — and 31.64% is Malayalam's EER even at its *own* best possible threshold, so this isn't a calibration problem that a per-language threshold would fix. The shared embedding genuinely doesn't separate real from fake well for Malayalam. Gujarati, by contrast, generalizes about as well as the trained languages. See §12 for the fix.

12 The fix: training directly on all 5 languages

§11 showed Malayalam breaking hard (65.37% accuracy, 31.64% EER) even at its own best threshold — a genuine feature-separability problem, not a calibration one. Before retraining, a quick diagnostic ruled out the obvious suspect.

❶ **Backbone-coverage check, ruled out.** A linear-probe test — 5-fold cross-validated logistic regression on frozen mean-pooled embeddings, no training involved — scored **98.67%** separability for the current XLSR backbone on a balanced Malayalam sample, versus **98.33%** for Meta's MMS-300m (a backbone pretrained explicitly on 1000+ languages, Malayalam included). Swapping backbones would not have helped: the frozen features already carried near-perfect real/fake signal for Malayalam. The bottleneck was never backbone coverage — it was that the conformer head had simply never been trained on a single Malayalam clip.

What changed

01 Re-cached frozen-backbone features for the full corpus

All 5 languages already sitting in `indic_df/manifest.tsv` (Gujarati and Malayalam had proper train/dev/test splits left over from the paused 7-language build, §4 — they'd just never been used for training) — 20,535 train / 4,770 dev / 3,884 test clips.

02 Retrained the same 1.79M-param conformer head

on `hi+bn+ta+ml+gu` jointly Identical architecture and training script as §5 — the only change is which languages are in the training set. Best checkpoint at epoch 11: dev EER 0.10%.

03 Re-ran the fixed-threshold confusion matrices from §11

on the held-out test split for every language New operating threshold (in-domain EER 0.16% on the `hi/bn/ta` test set) applied uniformly, exactly as before.

Before → after, the two languages that mattered

ML Malayalam

test split

ACCURACY BEFORE ACCURACY AFTER

65.37% **100.00%**

EER BEFORE EER AFTER

31.64% **0.00%**

GU Gujarati

test split

ACCURACY BEFORE ACCURACY AFTER

97.90% **100.00%**

EER BEFORE EER AFTER

1.28% **0.00%**

"Before" (§11) scored every clip of the then-unseen language, $n=3,846$ (ml) / 3,529 (gu).

"After" scores only the held-out test split — $n=326$ (ml) / 516 (gu) — since retraining

moved the rest into train/dev. Different n by necessity, to keep the comparison leak-free.

Full held-out test set, all 5 languages, this checkpoint

All 5 languages combined

$n=3,884$

		PRED. REAL	PRED. FAKE
ACTUAL	REAL	2,712	4
	FAKE	2	1,166

ACCURACY PRECISION RECALL EER
99.85% **99.93%** **99.85%** **0.16%**

BN Bengali

$n=692$

		REAL	FAKE
REAL	REAL	596	0
	FAKE	0	96

ACCURACY EER
100.00% **0.00%**

HI Hindi

$n=1,368$

		REAL	FAKE
REAL	REAL	748	4
	FAKE	0	616

ACCURACY EER
99.71% **0.30%**

TA Tamil

n=982

	REAL	FAKE
REAL	704	0
FAKE	2	276

ACCURACY EER
99.80% 0.39%

GU Gujarati

n=516

	REAL	FAKE
REAL	356	0
FAKE	0	160

ACCURACY EER
100.00% 0.00%

ML Malayalam

n=326

	REAL	FAKE
REAL	308	0
FAKE	0	18

ACCURACY EER
100.00% 0.00%

i The diagnosis held up. §11 argued Malayalam's failure was a training-data gap, not a backbone or calibration problem — folding its data into training fixed it completely (31.64%→0.00% EER) without hurting any other language (all five now sit at or below 0.39% EER on held-out test data). This is the strongest evidence in this report that the frozen-backbone architecture generalizes fine *once the head has actually seen the language*.

⚠ Trade-off: no held-out language remains. This checkpoint has now trained on every language in the corpus, so there is no longer any way to test true zero-shot cross-lingual generalization (what §10 measured) with the current data. Any claim about performance on a genuinely new, 6th language is untested, not just untested-and-good. §13 fixes that by pulling in 5 fresh ones.

13 Zero-shot test: 5 more languages, never in the corpus

§12 closed off the corpus's only held-out language, so the honest next step is to pull in languages that were never part of this project at all — new real-speech sources, scored with the same fixed deployment threshold, zero retraining.

01 5 new languages, 500 clips each

(250 real + 250 fake) Real speech from fresh HF sources per language (PharynxAI, an IIIT-B ASR corpus, a Punjabi ASR set, a Sanskrit ASR set) — none touched by this project before. Fake speech from the same vdivyasharma/IndicSynth generator family used throughout, so the fake side is apples-to-apples with every other result in this report.

02 Same checkpoint, same fixed threshold

(−1.0212, §12) No recalibration, no fine-tuning — the exact deployment setting from §12, pointed at languages it has genuinely never seen.

LANGUAGE	FAMILY	N	ACCURACY (FIXED THR.)	EER (OWI
KN Kannada	Dravidian	500	99.40%	
MR Marathi	Indo-Aryan	500	99.80%	
PA Punjabi	Indo-Aryan	500	98.40%	
TE Telugu	Dravidian	500	98.40%	
SA Sanskrit	Indo-Aryan (liturgical)	500	96.20%	

Mean across all 5: 98.44% accuracy, 1.04% EER. "Family" is linguistic lineage, not a corpus label — included because it's the most likely explanation for Sanskrit's outlier result below.

SA Sanskrit – the outlier

n=500

	REAL	FAKE
REAL	243	7
FAKE	12	238

ACCURACY EER
96.20% **4.80%**

i Zero-shot generalization holds up, mostly. 4 of 5 fresh languages — two Dravidian (Kannada, Telugu), two Indo-Aryan (Marathi, Punjabi) — land at 98.4–99.8% accuracy and 0.00–0.40% EER despite zero exposure during training. That's close to the trained-language numbers in §12, on languages this checkpoint has truly never touched. The frozen-backbone-plus-small-head recipe generalizes well within the modern-Indic-language distribution it was built on.

⚠ Sanskrit is the one language that visibly struggles (4.80% EER, 12 fakes slipped through as real out of 250). The likely cause isn't the language itself so much as a register mismatch: the real-speech source here is recited/liturgical Sanskrit (formal chanting cadence, sustained vowels), while every other language's "real" side in this report is spontaneous conversational speech. That's a corpus-distribution gap, not necessarily a Sanskrit-specific one — it would be worth re-testing with conversational Sanskrit speech, if a corpus of that exists, before concluding the model is weak on the language per se.

14 Limitations & honest caveats

- **Small dev/test sets per language.** Bengali dev is 454 clips, Tamil dev 258 — early-stopping decisions and the 0.00% Tamil EER numbers should be read as encouraging, not as statistically tight estimates.
- **Corpus scaled down from 6GB to ~2.9GB** mid-build (see §4) after streaming-resume costs made repeated interruptions expensive. More data, particularly more distinct TTS/VC systems, would likely improve robustness to attacks not represented here.
- **Only 12 languages' worth of "fake" generators are FreeVC24 and related IndicSynth models.** The fine-tuned head has not seen every deepfake technique in the wild — it has learned to catch the ones in this corpus extremely well.
- **Training crashed at epoch 5** on a CUDA out-of-memory error, likely triggered by a concurrent CPU-side evaluation job pushing the system over the edge while GPU VRAM was already pinned near its 8GB ceiling throughout. The epoch-4 checkpoint (already the best on dev EER) was kept as final rather than risk another crash chasing marginal gains.
- **Catastrophic forgetting on the original task** (§7) is real and unmitigated in this run.
- **The gated 12-language ai4bharat/IndicVoices dataset was not used** — access approval didn't register, and its audio needs FFmpeg tooling not present on this machine. This report covers 3 of India's 22 scheduled languages.
- **The domain-adversarial checkpoint (§9–10) is not a clear upgrade.** Its cross-lingual EER (12.50%) is statistically tied with the plain model's (12.38%) on the ml+gu test, and the dev-set language accuracy never cleanly converged to the 33.3% chance floor. Superseded in practice by §12: direct labeled data did what adversarial exposure aimed for and then some, so this is now a lower-priority avenue unless a genuinely unlabeled new language shows up.
- **The cross-lingual test itself was 2 languages, unevenly difficult** (Gujarati ~1.5–2% EER, Malayalam ~21% EER for both §9 checkpoints) — not enough to generalize "this works/doesn't work across languages" as a blanket claim. Moot now regardless: see next point.
- **Malayalam failed hard at its own best threshold (§11): 31.64% EER, 65.37% accuracy — fixed in §12** by folding Malayalam (and Gujarati) into training; a linear-probe diagnostic confirmed the frozen backbone's features were already separable (98.67%), so the fix was data, not architecture.
- **Sanskrit is the weak point among the 5 fresh zero-shot languages tested in §13** (4.80% EER vs 0.00–0.40% for Kannada/Marathi/Punjabi/Telugu). Likely a register mismatch (liturgical/recited speech vs. the conversational speech everywhere else in the corpus) rather than a language-specific failure, but that's a hypothesis, not a confirmed diagnosis — no conversational-Sanskrit source was tested.
- **The §13 zero-shot languages were tested at n=500 each** (250 real + 250 fake), and only against the IndicSynth generator family already used throughout this report. A larger sample

and a wider set of unseen deepfake generators would tighten these estimates and test a dimension (generator diversity) this report doesn't cover at all.

15 Reproducing this report

Every number above comes from a checked-in script and a file under `results/` or `results_indic/`.

FILE	
<code>evaluate_model.py</code>	
<code>build_indic_dataset.py</code>	
<code>cache_features.py</code>	
<code>finetune_indic.py</code>	
<code>evaluate_indic.py</code> / <code>confusion_indic.py</code>	
<code>model_indic_ft.safetensors</code>	
<code>finetune_language_agnostic.py</code>	
<code>evaluate_cross_lingual.py</code>	
<code>model_indic_ft_agnostic.safetensors</code>	
<code>confusion_all_languages.py</code>	Fixed-threshold confusion matrices a
<code>probe_mms_vs_xlsr.py</code>	
<code>model_indic_ft_5lang.safetensors</code>	
<code>confusion_all_languages_5lang.csv</code>	
<code>test_new_language.py</code>	§13's zero-sho
<code>eval_new_languages_all.csv</code>	

Full commands and narrative in `results_indic/README.md` and `results/RESULTS.md`.

XLSR-Mamba Field Report

Generated 11 Sep 2026, updated 13 Sep 2026 · RTX 5060 Laptop · PyTorch 2.10.0+cu128